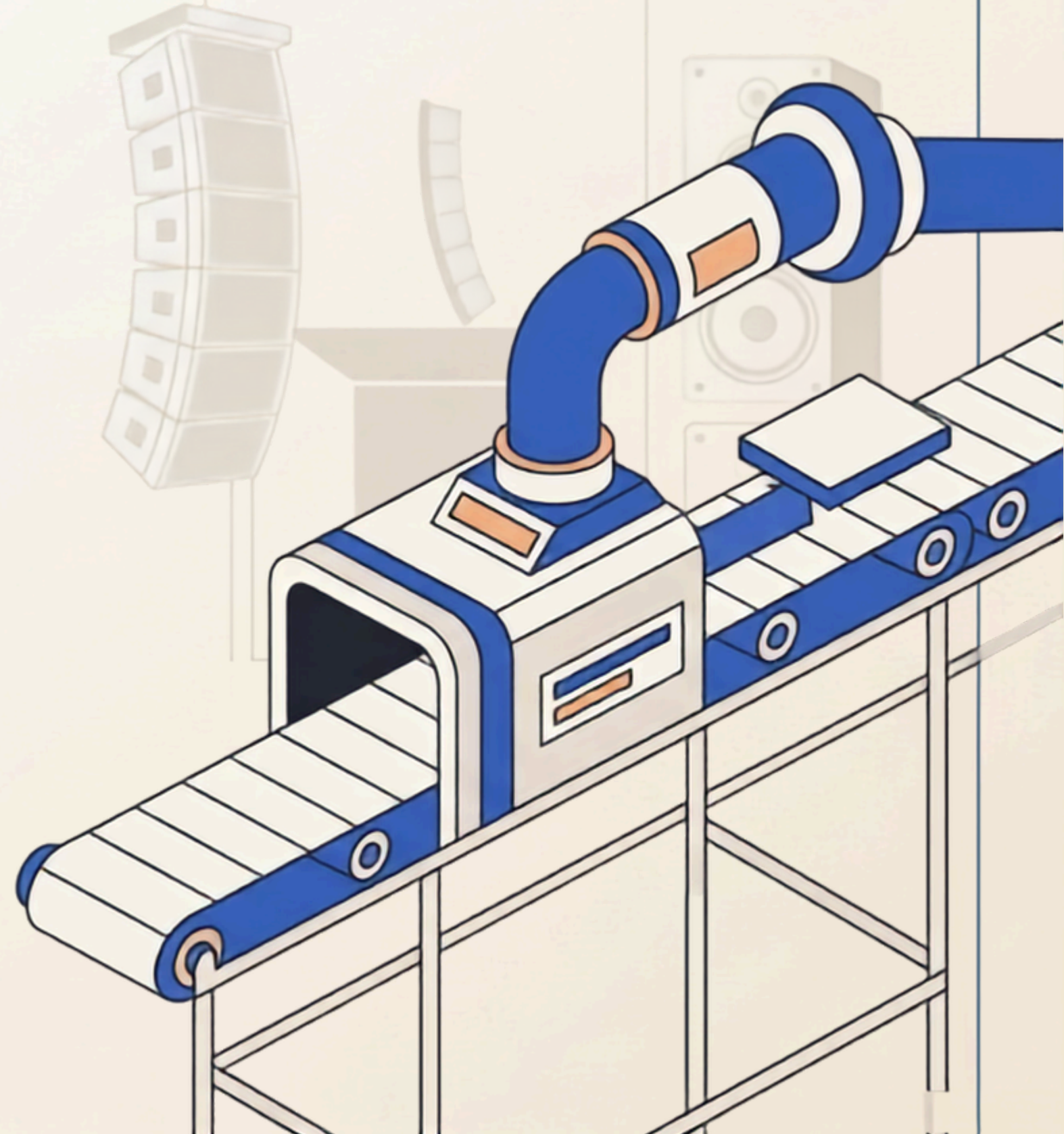


Functional Programming in Java: Streams API

Advanced Web Development

Members: Esteban Quiroga,
David Rodríguez,
Ángel Sabando

Case Study: Audio Equipment Inventory



What is Functional Programming and the Streams API?

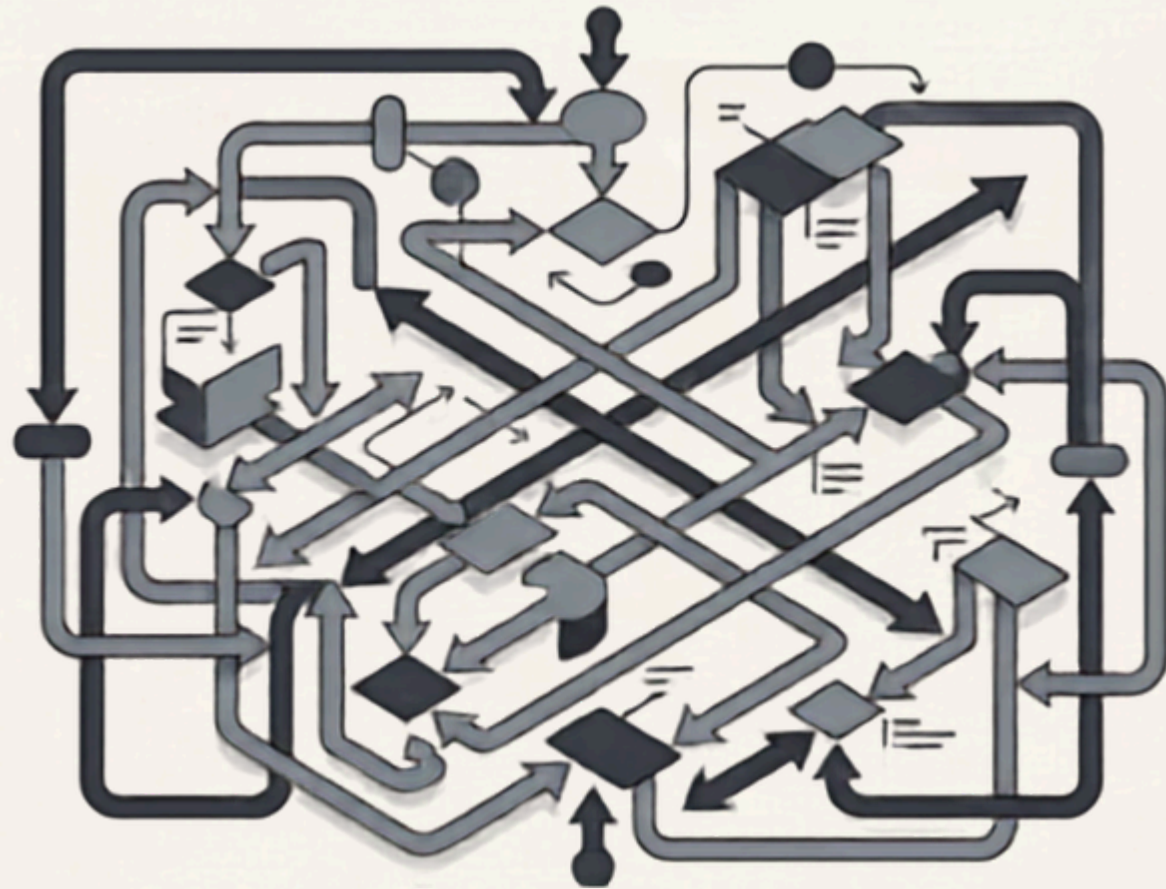
Functional Programming:

- Treats computation as function evaluation.
- Avoids mutable state and side effects (altering external variables, etc.).
- Emphasizes pure functions.

Streams API (Java 8):

- Enables declarative processing (describing “what” you want, not “how” step-by-step).
- Uses lambdas (short anonymous functions, e.g. $e \rightarrow e.\text{price()} > 1000$).
- Uses method references (shorthand for existing methods, e.g. `AudioEquipment::inStock`).

IMPERATIVE



DECLARATIVE

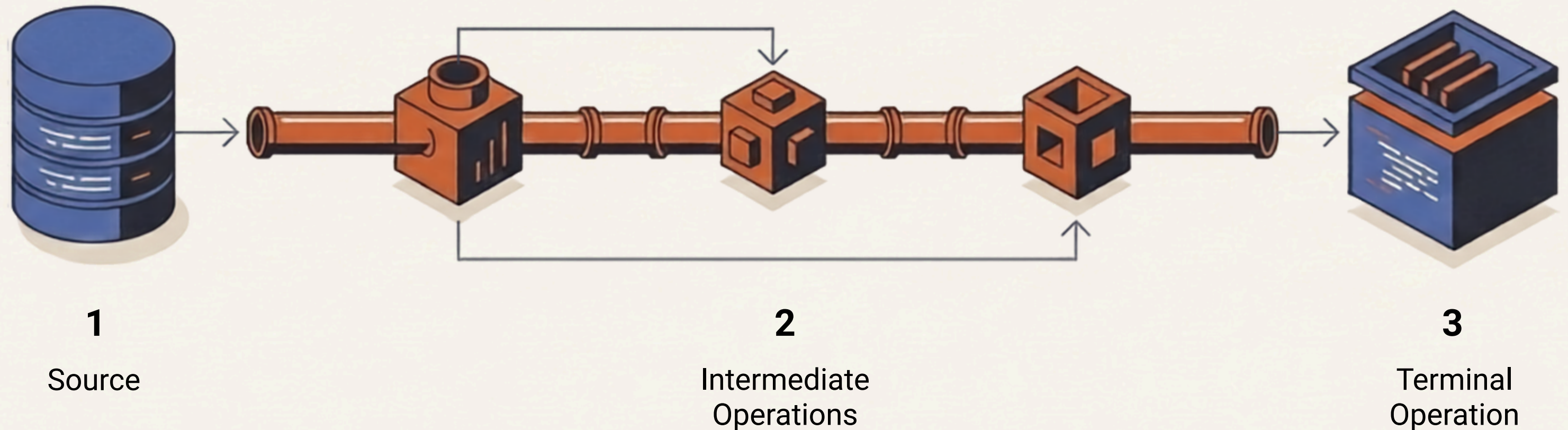


What is a Pipeline?

Start: The original collection (e.g., `inventory.stream()`).

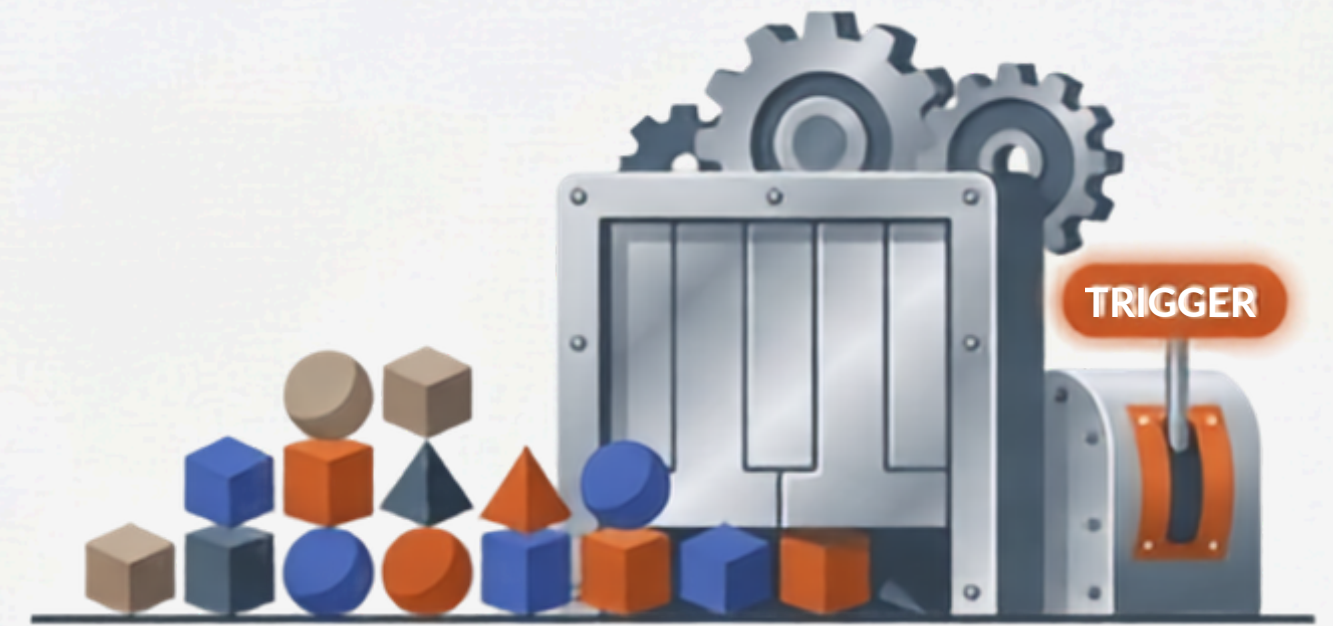
Lazy Processing: Chain of operations that transform the flow, but don't execute yet.

Execution & Result: Triggers the entire pipeline to run and produce the final outcome.



Intermediate Operations and Lazy Evaluation

Intermediate operations are lazy: Java doesn't process any element until a terminal operation is called; the pipeline is only “assembled,” not executed.



PIPELINE: ASSEMBLED, NOT EXECUTED



filter()): takes a predicate (a function that evaluates a condition, returning true or false) and selects matching elements



map()): transforms each element by applying a function, possibly changing its data type



sorted()): orders elements by natural order or a Comparator

Terminal Operations and Immutability

Terminal operations trigger actual processing and produce a concrete result.

collect(): gathers the result into a new list/set, without altering the original collection (immutability: the original data never changes)

mapToDouble() + sum(): converts to a primitive numeric stream, avoiding boxing/unboxing (the cost of converting between primitive **double** and its wrapper class **Double**)

forEach(): performs an action on each final element

Original Data



Result List



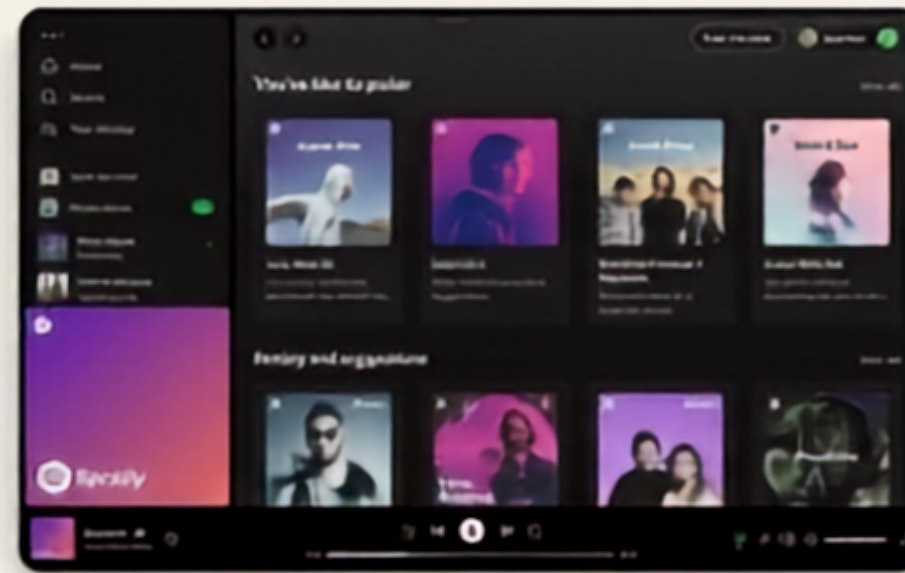
Real-world Applications of Stream-based Data Processing

NETFLIX



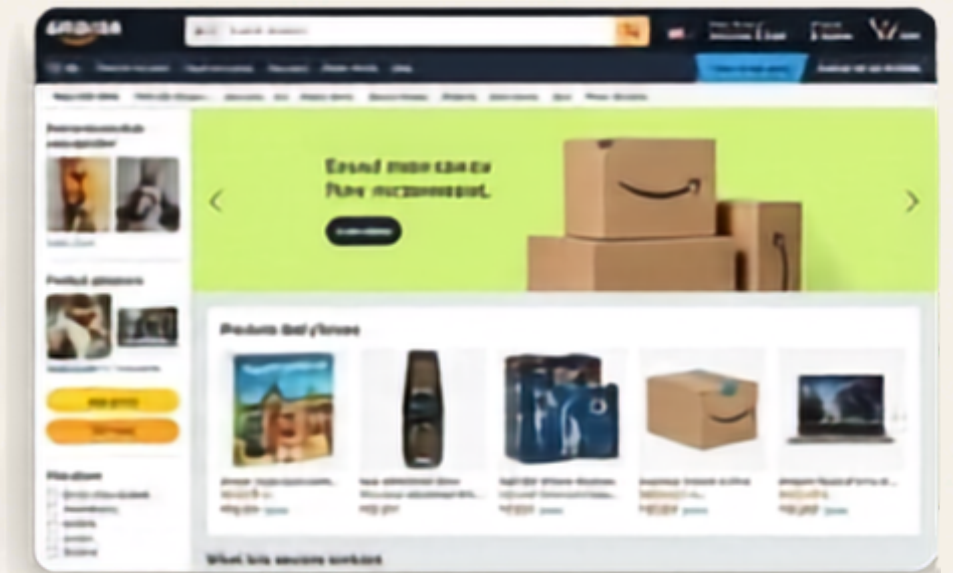
Real-time data processing for recommendations.

 **Spotify**



Live streaming analytics and suggestions.

amazon



Inventory and logistics flow management.

Benefits and Limitations of Java Streams

Benefits



Declarative style: focus on what to do, not how to loop through data



Internal iteration: the Stream API handles traversal for you



Lazy intermediate operations can improve efficiency by delaying execution until needed



Easy composition of filter, map, sort, and collect steps



Parallel streams can speed up some heavy data-processing workloads

Limitations



Streams are single-use; once a terminal operation runs, the stream cannot be reused



Debugging can be harder than with simple loops because logic is chained



Not ideal for very simple tasks where a loop is clearer



Side effects inside stream operations can make behavior harder to reason about



Parallel streams may hurt performance or create unpredictability if used incorrectly

Parallel Streams

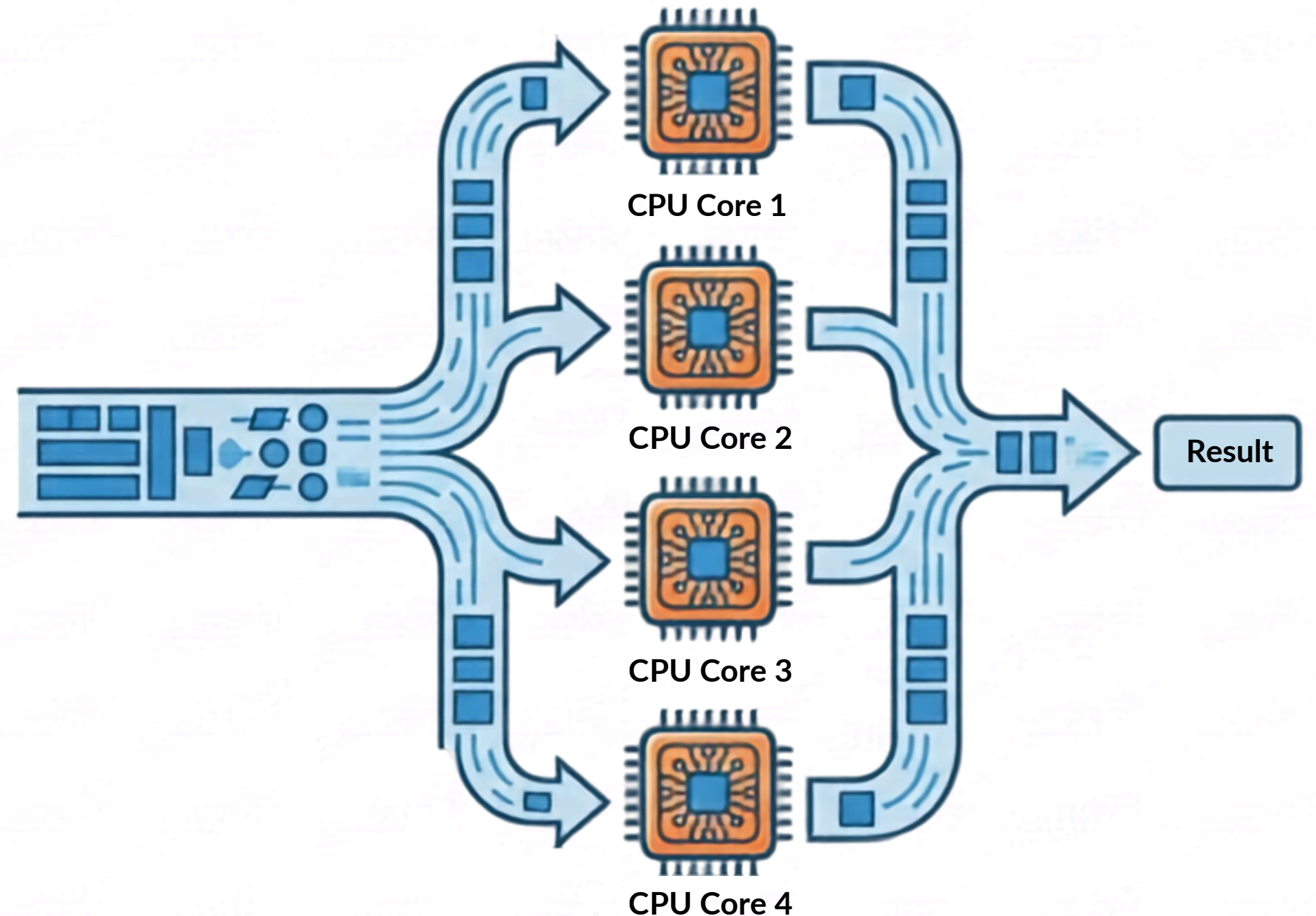
Use `.parallelStream()` to process data across multiple threads

Best for large datasets and CPU-bound tasks

Can improve performance by splitting work and combining results

Avoid it for small collections, I/O tasks, or shared mutable state

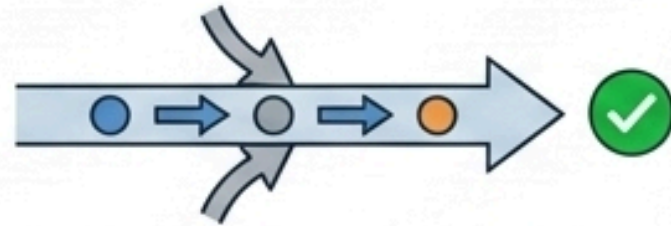
Always benchmark before assuming it is faster



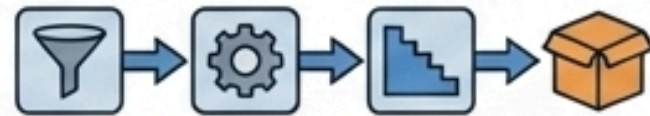
Best Practices and Anti-patterns

✓ Best Practices

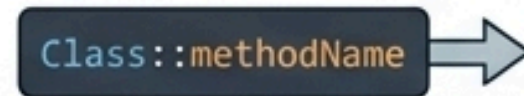
- Keep stream operations stateless and predictable



- Use streams for filtering, mapping, sorting, and collecting



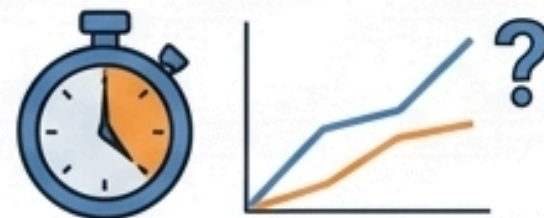
- Prefer method references when they improve readability



- Favor immutable data and clear transformation steps

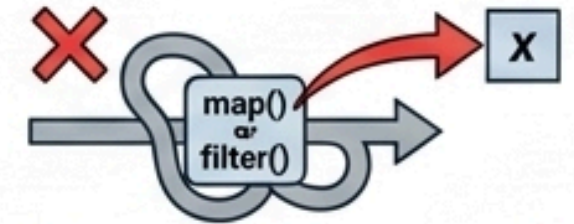


- Measure performance before using parallel streams



✗ Anti-patterns

- Add side effects inside `map()` or `filter()`



- Build overly long and hard-to-read pipelines



- Use `peek()` to modify elements instead of inspecting them



- Force streams when a simple loop is easier to understand



- Assume parallel streams are always faster



The Problem and Core Code

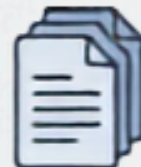
An import company needs: 'Line Array'/'Subwoofer' items in stock, price > \$1000, sorted alphabetically, plus total investment. Modeled with a record (an immutable Java class, ideal for the functional paradigm).



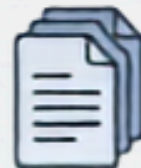
```
List<String> reportNames = inventory.stream()
    .filter(AudioEquipment::inStock)
    .filter(e -> e.type().equals("Line Array") ||
        e.type().equals("Subwoofer"))
    .filter(e -> e.price() > 1000.00)
    .map(AudioEquipment::brandModel)
    .sorted()
    .collect(Collectors.toList());
```

```
double totalInvestment = inventory.stream()
    .filter(AudioEquipment::inStock)
    .filter(e -> e.type().equals("Line Array") ||
        e.type().equals("Subwoofer"))
    .filter(e -> e.price() > 1000.00)
    .mapToDouble(AudioEquipment::price)
    .sum();
```


References and Further Learning



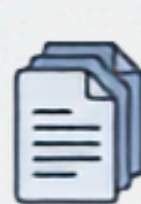
Oracle Java Documentation — Stream API: <https://docs.oracle.com/javase/8/docs/api/java/util/stream/Stream.html>



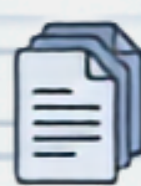
Oracle Dev Java — The Stream API Learning Path: <https://dev.java/learn/apistreams/>



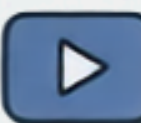
Oracle Java Package Documentation — java.util.stream:
<https://docs.oracle.com/javase/8/docs/api/java/util/stream/package-summary.html>



Beldung — When to Use a Parallel Stream in Java: <https://www.baeldung.com/java/when-to-use-parallel-stream>



HowToDoInJava — Internal vs. External Iteration in Java:
<https://howtodoinjava.com/java8/internal-vs-external-iteration/>



Complementary video: https://youtu.be/tklkyVa7KZo?si=uob5bW5_RqmuOxVh